9

SOFT ACTOR-CRITIC

SOFT ACTOR-CRITIC (SAC)

- Fue presentado en 2018 con el título: “Soft Actor-Critic: Aprendizaje por refuerzo profundo de entropía máxima off-policy con un actor estocástico” (Haarnoja et al., 2018).
- Fue diseñado para ser eficiente, estable y adecuado para entornos continuos.
- SAC busca no solo maximizar la recompensa esperada, sino también maximizar la entropía de la política (fomenta políticas más exploratorias y robustas).

Soft Actor-Critic:
Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor

Tuomas Haarnoja [1] Aurick Zhou [1] Pieter Abbeel [1] Sergey Levine [1]

rXiv:1801.01290v2 [cs.LG] 8 Aug 2018

Abstract

Model-free deep reinforcement learning (RL) algorithms have been demonstrated on a range of challenging decision making and control tasks. However, these methods typically suffer from two major challenges: very high sample complexity and brittle convergence properties, which necessitate meticulous hyperparameter tuning. Both of these challenges severely limit the applicability of such methods to complex, real-world domains. In this paper, we propose soft actor-critic, an off-policy actor-critic deep RL algorithm based on the maximum entropy reinforcement learning framework. In this framework, the actor aims to maximize expected reward while also maximizing entropy. That is, to succeed at the task while acting as randomly as possible. Prior deep RL methods based on this framework have been formulated as Q-learning methods. By combining off-policy updates with a stable stochastic actor-critic formulation, our method achieves state-of-the-art performance on a range of continuous control benchmark tasks, outperforming prior on-policy and off-policy methods. Furthermore, we demonstrate that, in contrast to other off-policy algorithms, our approach is very stable, achieving very similar performance across different random seeds.

of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL methods are notoriously expensive in terms of their sample complexity. Even relatively simple tasks can require millions of steps of data collection, and complex behaviors with high-dimensional observations might need substantially more. Second, these methods are often brittle with respect to their hyperparameters: learning rates, exploration constants, and other settings must be set carefully for different problem settings to achieve good results. Both of these challenges severely limit the applicability of model-free deep RL to real-world tasks.

One cause for the poor sample efficiency of deep RL methods is on-policy learning: some of the most commonly used deep RL algorithms, such as TRPO (Schulman et al., 2015), PPO (Schulman et al., 2017b) or A3C (Mnih et al., 2016), require new samples to be collected for each gradient step. This quickly becomes extravagantly expensive, as the number of gradient steps and samples per step needed to learn an effective policy increases with task complexity. Off-policy algorithms aim to reuse past experience. This is not directly feasible with conventional policy gradient formulations, but is relatively straightforward for Q-learning based methods (Mnih et al., 2015). Unfortunately, the combination of off-policy learning and high-dimensional, nonlinear function approximation with neural networks presents a major challenge for stability and convergence (Bhatnagar et al.,

CONTRIBUCIONES

- ✓ Combina lo mejor de Actor-Critic y maximización de entropía (fomenta políticas estocásticas).
- ✓ Algoritmo Off-Policy: Usa un replay buffer para eficiencia en muestreo (como DQN, pero para políticas continuas).
- ✓ Estabilidad y rendimiento mejor que DDPG, PPO y TD3 en entornos continuos (MuJoCo, robótica).

rXiv:1801.01290v2 [cs.LG] 8 Aug 2018

Soft Actor-Critic:
Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor

Tuomas Haarnoja [1] Aurick Zhou [1] Pieter Abbeel [1] Sergey Levine [1]

Abstract

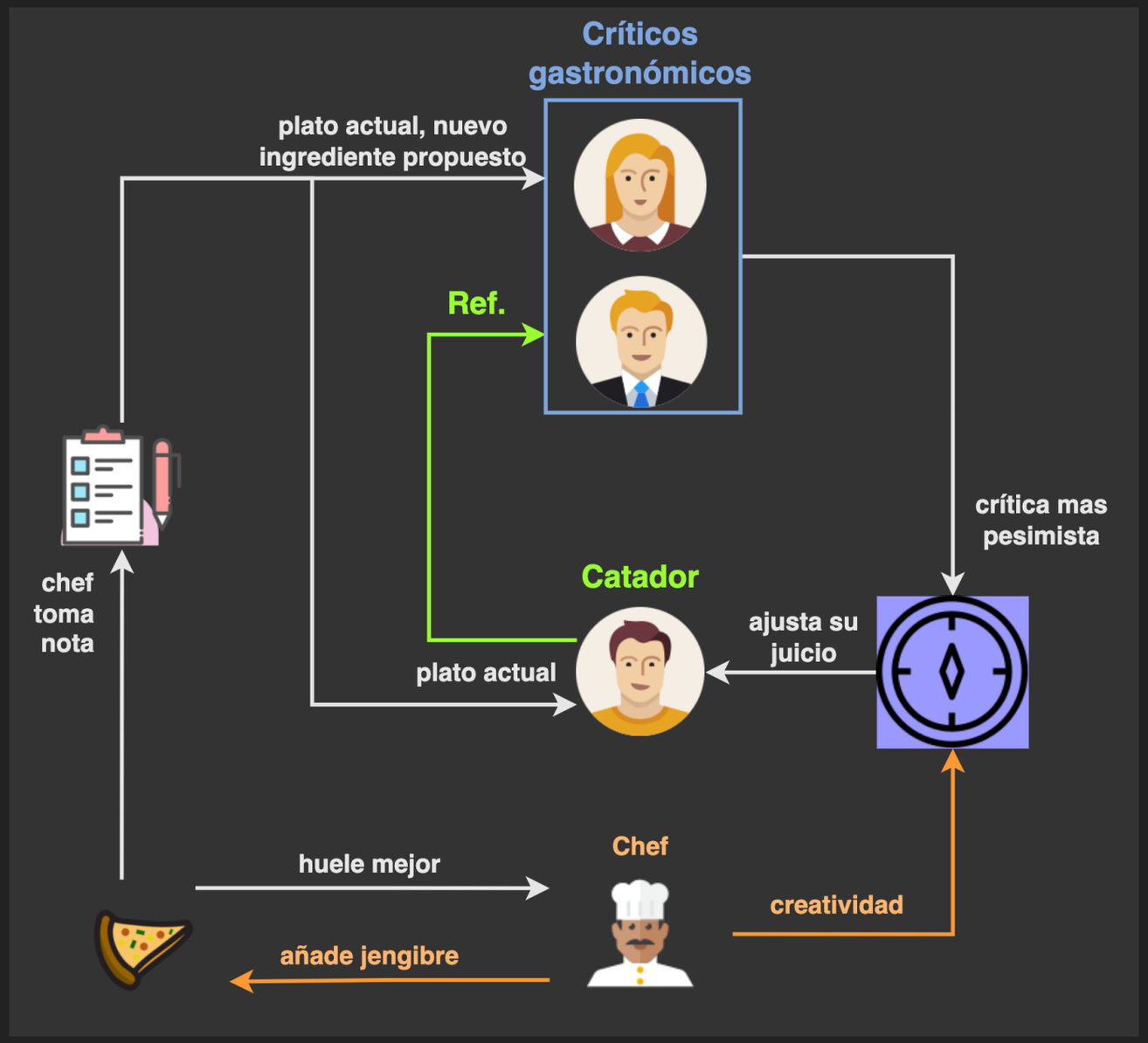
Model-free deep reinforcement learning (RL) algorithms have been demonstrated on a range of challenging decision making and control tasks. However, these methods typically suffer from two major challenges: very high sample complexity and brittle convergence properties, which necessitate meticulous hyperparameter tuning. Both of these challenges severely limit the applicability of such methods to complex, real-world domains. In this paper, we propose soft actor-critic, an off-policy actor-critic deep RL algorithm based on the maximum entropy reinforcement learning framework. In this framework, the actor aims to maximize expected reward while also maximizing entropy. That is, to succeed at the task while acting as randomly as possible. Prior deep RL methods based on this framework have been formulated as Q-learning methods. By combining off-policy updates with a stable stochastic actor-critic formulation, our method achieves state-of-the-art performance on a range of continuous control benchmark tasks, outperforming prior on-policy and off-policy methods. Furthermore, we demonstrate that, in contrast to other off-policy algorithms, our approach is very stable, achieving very similar performance across different random seeds.

of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL methods are notoriously expensive in terms of their sample complexity. Even relatively simple tasks can require millions of steps of data collection, and complex behaviors with high-dimensional observations might need substantially more. Second, these methods are often brittle with respect to their hyperparameters: learning rates, exploration constants, and other settings must be set carefully for different problem settings to achieve good results. Both of these challenges severely limit the applicability of model-free deep RL to real-world tasks.

One cause for the poor sample efficiency of deep RL methods is on-policy learning: some of the most commonly used deep RL algorithms, such as TRPO (Schulman et al., 2015), PPO (Schulman et al., 2017b) or A3C (Mnih et al., 2016), require new samples to be collected for each gradient step. This quickly becomes extravagantly expensive, as the number of gradient steps and samples per step needed to learn an effective policy increases with task complexity. Off-policy algorithms aim to reuse past experience. This is not directly feasible with conventional policy gradient formulations, but is relatively straightforward for Q-learning based methods (Mnih et al., 2015). Unfortunately, the combination of off-policy learning and high-dimensional, nonlinear function approximation with neural networks presents a major challenge for stability and convergence (Bhatnagar et al.,

ANALOGÍA

- El chef innovador.



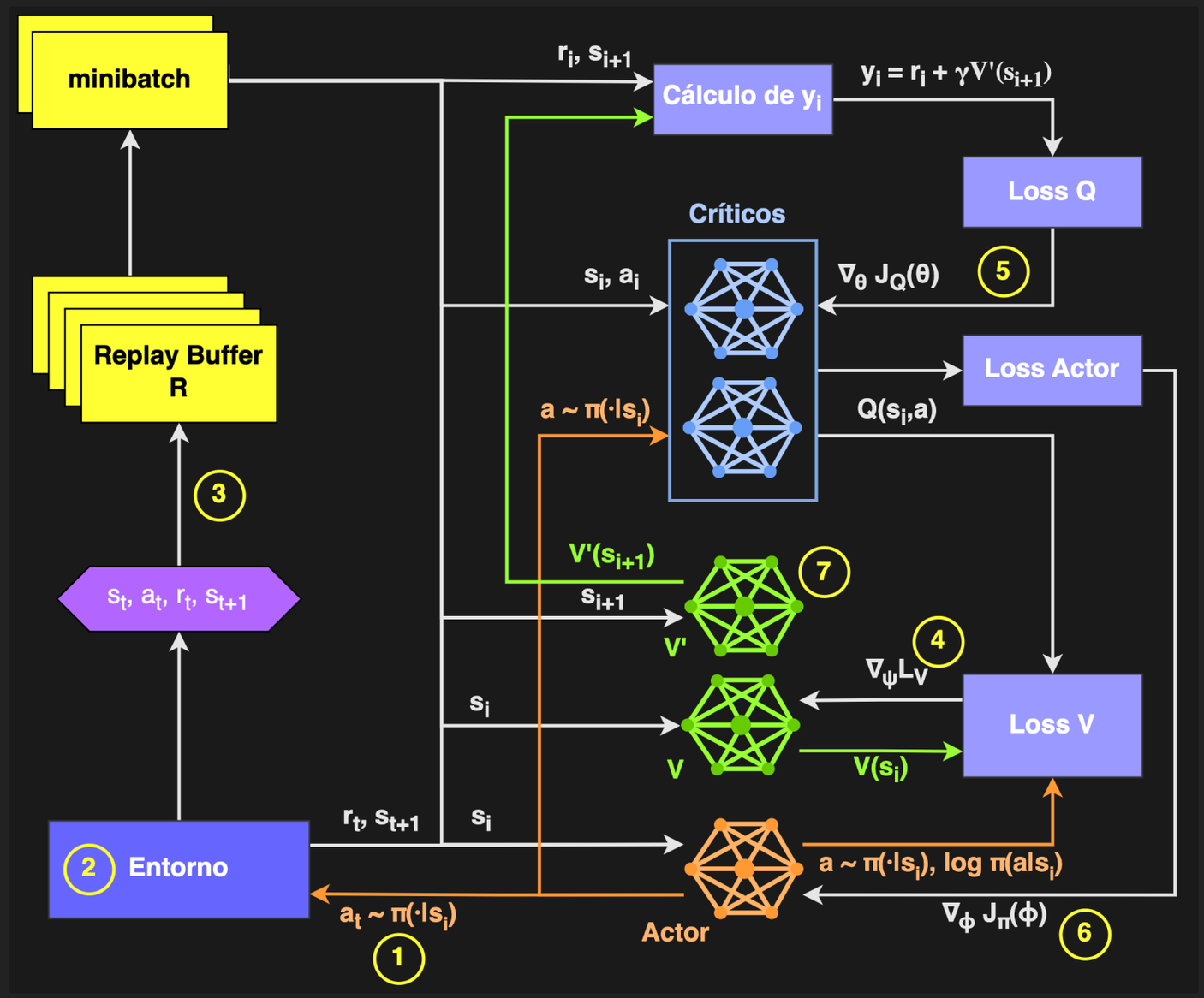
CARACTERÍSTICAS DE SAC

- Maximizar la recompensa + maximizar la entropía de las acciones.
- La entropía mide que tan aleatoria o impredecible es la política => mayor exploración => menos probabilidad de atasco.
- Es off-policy, esto es, utiliza un buffer de repetición => aprende de los datos antiguos.
- Evita la inestabilidad de DDPG:
  - ✓ Usa un actor estocastico (probabilidad de que acción tomar).
  - ✓ El mismo objetivo de maximizar la entropía funciona como un suavizador del aprendizaje (evita cambios bruscos).
  - ✓ Usa dos funciones Q para evitar sobrestimar los valores por ser optimista (dos críticos, similar a TD3).
- Se hicieron pruebas quitando partes (estudios de ablación).
- SAC es sensible a los hiperparámetros => temperatura (ajusta cuánta importancia le otorga a la entropía).

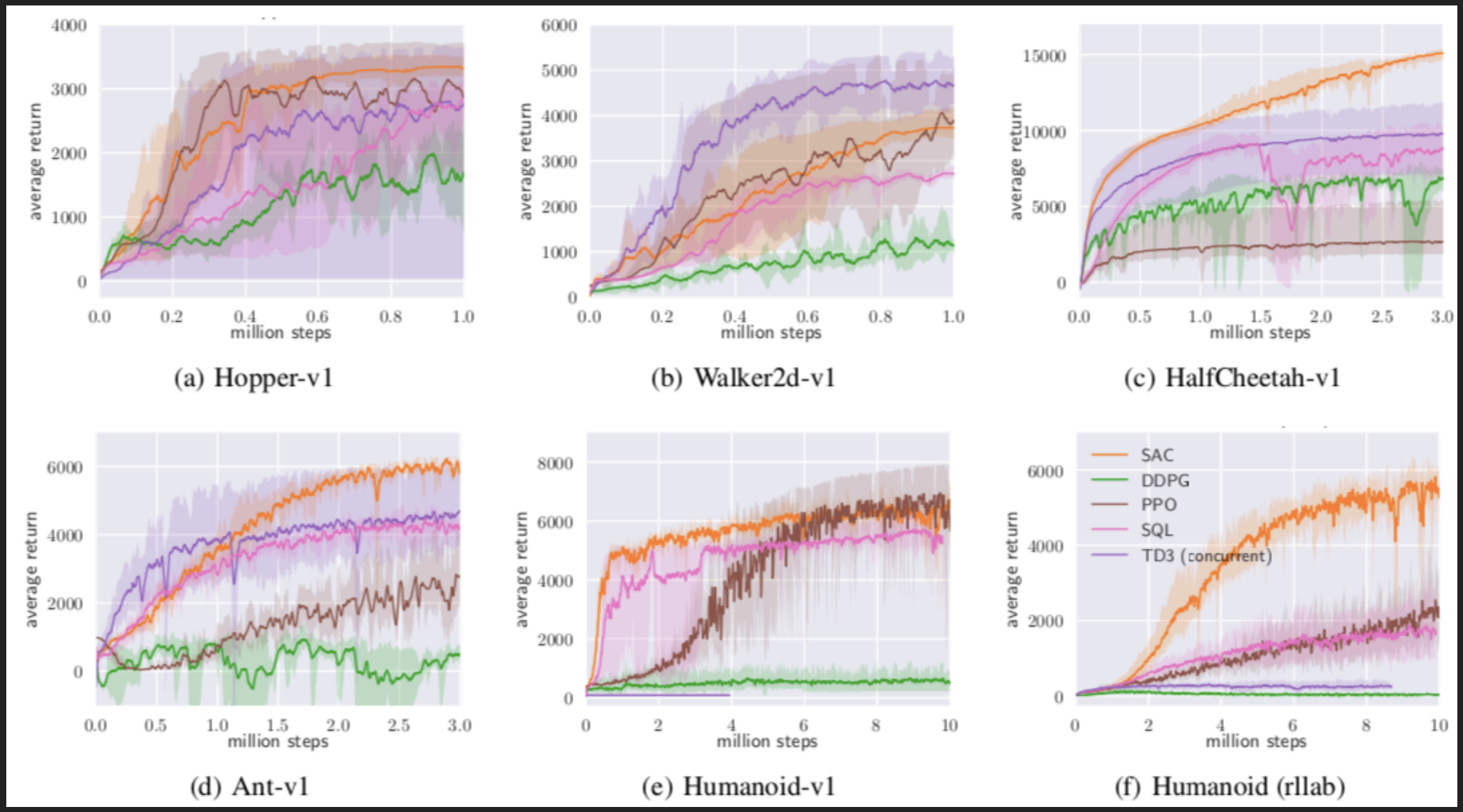
PSEUDOCÓDIGO Y ARQUITECTURA DE SAC

Algorithm 1 Soft Actor-Critic

Initialize parameter vectors ψ, $\bar{\psi}$, θ, ϕ.
for each iteration **do**
  for each environment step **do**
    1 $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$
    2 $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$
    3 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
  end for
  for each gradient step **do**
    4 $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$
    5 $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
    6 $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    7 $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$
  end for
end for

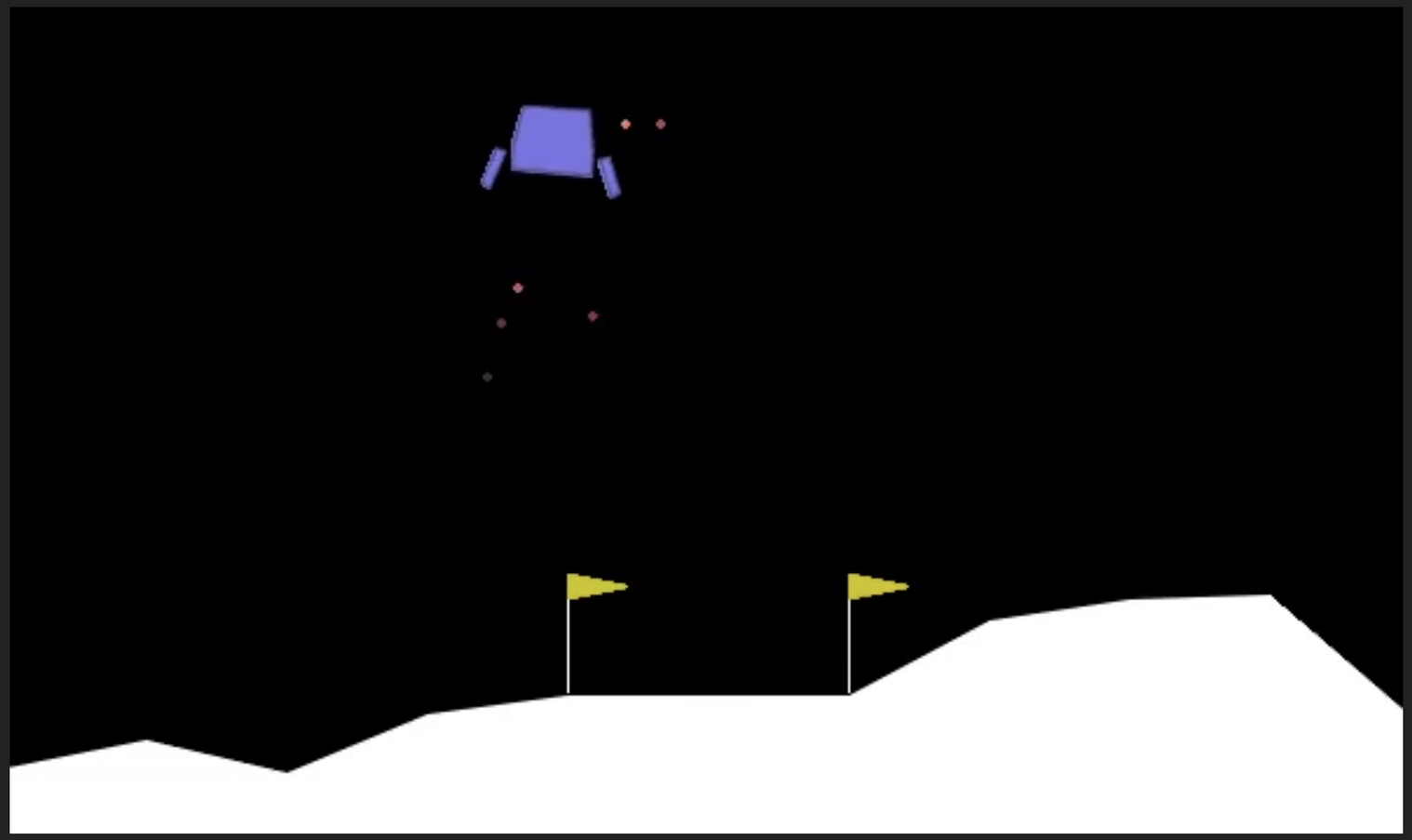


ENSAYOS CON SAC



EJEMPLO

- Aprendizaje del alunizaje con LunarLanderContinuous de Gymnasium y con la biblioteca de agentes Stable Baseline 3.



SB3

- learning_starts. Número de pasos de interacción con el entorno antes de comenzar el aprendizaje. Ayuda a llenar el buffer con experiencias variadas antes de actualizar la red.
- tau. Parámetro de interpolación suave para la actualización de las red objetivo V' (target network):
  - ✓ target_param = tau * param + (1 - tau) * target_param
  - ✓ Un valor pequeño da actualizaciones más estables.
- train_freq. Frecuencia con la que se entrena el modelo (en pasos del entorno), 1 significa que se entrena después de cada paso.
- gradient_steps. Número de pasos de gradiente a realizar por cada llamada de entrenamiento. Generalmente igual a train_freq para un entrenamiento equilibrado.

```
model = SAC(
    policy='MlpPolicy',
    env=train_env,
    learning_rate=3e-4,
    buffer_size=1000000,
    learning_starts=10000,
    batch_size=256,
    tau=0.005,
    gamma=0.99,
    train_freq=1,
    gradient_steps=1,
    verbose=1,
    seed=42
)
```

SB3 – EJEMPLO 1 (TRAIN_FREQ Y GRADIENT_STEP)

model = SAC("MlpPolicy", env, train_freq=4, gradient_steps=1, ...)

Paso de Entorno 1: Agente interactúa. Transición (s1, a1, r1, s2) se guarda en el replay buffer.

Paso de Entorno 2: Agente interactúa. Transición (s2, a2, r2, s3) se guarda.

Paso de Entorno 3: Agente interactúa. Transición (s3, a3, r3, s4) se guarda.

Paso de Entorno 4: Agente interactúa. Transición (s4, a4, r4, s5) se guarda.

train_freq alcanza el valor seteado , o sea 4 => se detiene la recolección de datos por un momento y se inicia la fase de entrenamiento:

Se extraerá un mini-batch del replay buffer.

Se realizará 1 paso de gradiente (porque gradient_steps=1).

Después de este paso de gradiente, el agente vuelve a interactuar con el entorno.

SB3 – EJEMPLO 2 (TRAIN_FREQ Y GRADIENT_STEP)

model = SAC("MlpPolicy", env, train_freq=4, gradient_steps=5, ...)

Paso de Entorno 1, 2, 3, 4: Agente interactúa, datos se guardan.

train_freq=4 alcanza el valor seteado. Se inicia la fase de entrenamiento:

Paso de Gradiente 1: Se extrae mini-batch A, se calculan pérdidas, se actualizan pesos.

Paso de Gradiente 2: Se extrae mini-batch B (diferente de A), se calculan pérdidas, se actualizan pesos.

Paso de Gradiente 3: Se extrae mini-batch C, se calculan pérdidas, se actualizan pesos.

Paso de Gradiente 4: Se extrae mini-batch D, se calculan pérdidas, se actualizan pesos.

Paso de Gradiente 5: Se extrae mini-batch E, se calculan pérdidas, se actualizan pesos.

gradient_steps=5 , o sea, alcanza el valor seteado. Termina la fase de entrenamiento.

El agente vuelve a interactuar con el entorno.

SOFT ACTOR-CRITIC V2 (SAC-V2)

- Una nueva versión fue presentada en 2019 con el título: "Soft Actor-Critic: Algorithms and Applications" (Haarnoja et al., 2019).

- Mejora: Autoajuste de la temperatura de entropía (α).

REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018, July). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In International conference on machine learning (pp. 1861-1870). Pmlr.
- Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., & Levine, S. (2019). Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905.